ZK 专家养成课程 · 大师篇（M00–M08）

> 本卷由 9 讲合并装订而成（内容与单文件版逐字一致）。篇内各讲以 `# S/L/M xx` 标题分界，总纲见 `README_课程总纲.md`。

本卷目录

- **M00** · M00 · 导师培养方案：从入门到行业顶尖
- **M01** · M01 · 专家地图：证明系统的设计空间
- **M02** · M02 · FRI 与哈希承诺家族
- **M03** · M03 · 数域的选择学：为什么新一代系统都在"换钟"
- **M04** · M04 · 查表论证全景：从 Plookup 到 Lasso/Jolt
- **M05** · M05 · 折叠与递归：Nova 一族
- **M06** · M06 · 安全与攻击者视角：健全性预算、真实攻击与审计清单
- **M07** · M07 · 从论文到代码：实现方法论
- **M08** · M08 · 顶石设计：给陌生负载造一套完整证明系统（出师关）

M00 · 导师培养方案：从入门到行业顶尖

> 这是大师篇的总纲。回答一个问题：**一个 ZK 领域的顶级导师，会怎样展开教学，才能把学生带到"行业顶尖大牛"的水准？** 前提盘点：学生已完成（或正在完成）基础篇 S01–S06、提高篇 S07–S10、论文精读 S11。这套课程把学生带到了哪里、还差什么、怎么补——本方案全部说清。

---

1. 先定义终点："行业顶尖"的五级阶梯

"精通 ZK"是句空话，顶尖必须拆成**可检验的等级**。每一级都有明确的过关物，导师据此判断是否放行：

级别	能力定义	过关物（可检验）	课程落点
L1 会算	任何协议都能在纸面跑通小例子，每个中间数可复核	mod-13 手跑 sumcheck / KZG 打开 / LogUp 折叠，零算错	S01–S06、S07–S09

级别	能力定义	过关物（可检验）	课程落点
L2 会证	白板证明完备性/健全性；会给一个系统做"健全性预算表"	10 分钟白板：sumcheck RBR 论证、三点恒等式、LogUp 恒等式	S06 §5、S08 §4
L3 会读	新论文 1 小时抽出协议骨架；任何实现 30 分钟定位关键路径	S11 式的逐节伴读报告；prove_chunk 盲测定位	S11、L07
L4 会造	从论文到生产级实现（对拍/fuzz/上游 PR）；为给定负载设计完整协议栈	P11 期末项目被合并；M08 顶石设计通过评审	M07、M08
L5 会评/会创	批判性评估（找出论文的隐含假设与薄弱环节）；提出改进并用实验验证；观点被同行引用	协议审计报告（含攻击）；ePrint 短文或开源工具	M06 + 研究循环

顶尖大牛的日常标志（面试/评审现场可检验）：给一个陌生工作负载，30 分钟内给出系统选型并量化理由；被同行挑战时能当场推导，而不是引用权威。**L4 之前靠课程，L4→L5 靠实战循环**——这是导师方案的核心结构。

2. 导师的六个教学动作（怎么教才到得了 L4/L5）

1. **苏格拉底链**：每个新概念，导师先连问三个问题再开讲（"验证方为什么不能自己算这个？""如果不承诺会发生什么？""这个步骤去掉会破坏哪条性质？"）。现有 S 课的"先做后名"就是它的书面化。
2. **先做后名**：已在 S 课执行；M 阶段升级为"**先实现后读论文**"——M02 让学生先手跑 mini-FRI，再去读 FRI 原文。
3. **费曼回讲**：W3/W6/W12 三个检查点，学生 10 分钟白板讲透一个证明，导师只追问不提示。讲不清 = 回炉。
4. **失败解剖**：导师**故意**给出有漏洞的协议变体（如去掉 batched sumcheck 的 γ 随机性、eq 表预先合并），学生必须找到攻击。L4/L5 的核心训练。
5. **数字强迫症**：任何主张必须配可复算数据。本课程 v1.1 审计（域常数错、健全性口径错）就是示范——大牛的第一反射是"这个数从哪来"。
6. **真实战场**：L4/L5 不接受玩具闭环——对拍要对着仓库 CPU golden、PR 要提给真实上游、评审要模拟 ePrint 的匿名恶评。

3. 缺口诊断：S01–S11 之后还差什么

能力维度	S 课已覆盖	顶尖要求的缺口	对应模块
IOP 原语	sumcheck/GKR 式/LogUp-GKR（一个家族的深度）	看见**整个 IOP 设计空间**的广度	M01
承诺方案	KZG/HyperKZG（配对族）	**哈希族**（FRI/BaseFold/WHIR/Binius）与**内积族**；选型判断	M02
数域	BN254 Fr 一个域	小域（Goldilocks/M31/二进制塔）为何主导新一代系统	M03
查表	LogUp-GKR 一个点	Plookup→Caulk→cq→Baloo→Lasso/Jolt 谱系与取舍	M04
折叠/递归	无	Nova 一族；folding×sumcheck 混合；IVC/聚合	M05
安全	健全性直觉	FS 的 ROM 现状、grinding 攻击、健全性预算表、审计清单	M06
工程闭环	对拍工具、算子规格	从论文到上游 PR 的完整方法论	M07
设计	一个系统的架构	给陌生负载做全套设计并答辩	M08

结论：S 课是"L1–L3 的最佳路径"，但单靠它会产出"DeepProve 专家"而非"ZK 大牛"。大师阶段 = M-track 八模块 + 研究循环。

4. M-track 大纲（每模块：目标 / 内容 / 动手 / 过关）

模块	目标（一句话）	动手任务	过关标准
M01 设计空间总图 ✅	脑里有"算术化 × PCS × 查表 × 折叠"四轴地图，能做选型	给 4 个工作负载做选型辩护	导师扮演恶意评审，挑不出论证漏洞

模块	目标（一句话）	动手任务	过关标准
M02 FRI 与哈希承诺 ✅	手跑 mini-FRI；讲清 BaseFold/WHIR/Binius 与 KZG 族的硬取舍	mod-13 mini-FRI 全程 + Rust 版	三点抽查攻击实验 + 与论文对拍
M03 数域的选择学	为什么 Goldilocks/M31/二进制塔统治新系统；2-adicity 与 FFT 的关系；BN254 的取舍	在 mod-13 框架下推导"小域的代价"（位宽不足→扩展域）	选型报告：给 3 种工作负载配域
M04 查表论证全景	Plookup→Lasso/Jolt 谱系；LogUp-GKR 的坐标	实现 Plookup toy 并与 LogUp toy 对比证明大小	复杂度对比表（承诺/证明/验证三维）
M05 折叠与递归	Nova 的 relaxed R1CS 折叠直觉；folding×sumcheck 混合；IVC/聚合的区别	手推 Nova 对 2 约束电路的一步折叠	给"流式证明长视频"设计折叠方案
M06 安全与攻击者视角	FS 的 ROM 证明现状；grinding；健全性预算表；审计方法	攻击一个被故意弱化的协议（导师提供）	审计报告：列出攻击面 + 修复建议
M07 从论文到代码	论文→claim 卡片→最小实现→fuzz 对拍→上游 PR	把 S11 精读的协议之一实现并提 PR	PR 被合并或收到实质性 review
M08 顶石设计	给陌生负载（EVM 块 / 70B 模型 / 隐私交易）做全套设计	15 页设计文档 + 30 分钟答辩	三位"模拟评审"（导师出恶评）通过

排期：M01/M02 本次已交付；M03–M06 按批续写（每批 2 讲，同风格：先做后名 + mod-13 贯穿 + 数字可复算）；M07/M08 是实践模块，讲义只提供方法论与模板。

5. 大师阶段节奏（W13–W24，第二学期）

周	内容	产出
W13–W14	M01 地图 + M02 FRI	选型辩护 1 + mini-FRI
W15–W16	M03 数域 + M04 查表全景	选型报告 2 + Plookup toy

周	内容	产出
W17–W18	M05 折叠	Nova 一步手推 + 折叠设计
W19–W20	M06 安全	攻击实验 + 审计报告
W21–W22	M07 论文→代码	上游 PR
W23–W24	M08 顶石	设计文档 + 毕业大答辩（三位模拟评审）

与研究循环并行（每周固定 4 小时）：ePrint 每周筛读 1 篇（L09 §3 方法论）→ 每月一篇精读笔记 → 每季度一个开源贡献。**L5 不是课程能教的，是循环里长出来的**——导师的职责是保证循环不断。

6. 出师标准（毕业大答辩的评分表）

关卡	形式	及格线
白板关	抽签复述任一核心证明（sumcheck RBR / 三点 / LogUp / Nova 折叠）	无逻辑跳跃，能应答两层追问
实现关	P2/P5/P7/P11 的代码现场改需求	30 分钟内改对且对拍仍绿
设计关	M08 设计文档答辩	选型有量化依据；对"为什么不选 X"有答案
评审关	导师出示一篇真实 ePrint 论文	60 分钟内产出 claim 卡片 + 两个质疑

四关全过 = 出师。此后学生与导师的关系从"教学"变为"同行"——一起读论文、互评设计、共挑协议毛病。

7. 导师的三条底线

1. **诚实优先于权威**：讲义错了就承认并修（v1→v2、v1.1 审计都是先例）；学生发现错误同等感谢；
2. **一切可复算**：没有源码行号或推导的数字不进讲义；
3. **不装懂**：超出确认范围的内容明确标注边界（如 S11 对附录 B 的处理），并给学生的自查路径。

下一讲：M01_专家地图——四轴设计空间。这是 L4 的第一块肌肉：顶尖大牛脑里那张"任何系统放进来都能定位"的地图。

M01 · 专家地图：证明系统的设计空间

> 顶尖大牛与熟练工程师的区别，是脑里有没有一张**能给任何系统定位**的地图。本讲把地图画出来：四根轴——**算术化**（把计算变成代数对象的方式）、**多项式承诺**（锁数据的方式）、**查表**（处理非线性）、**折叠**（摊销与递归）。每根轴先给谱系，再给取舍，最后标出 DeepProve 的坐标——以及**它在哪些场景会输**（会画地图的人必须知道自己地图的边界）。 前置：S06（sumcheck）、S07/S08（承诺）、S09（查表）、S10（系统）。所有"点"的一句话定位都可在 L09 资料清单找到论文出处。

---

0. 地图怎么用（一个真实场景）

面试题："我们要证明一个 70B 模型的每次推理输出，验证方是手机。" 顶尖的答案是先问四个问题——**每个问题对应地图上一根轴**：

1. 计算长什么样、什么算子占大头？（→ 轴一 算术化）
2. 验证方要多便宜、要不要 trusted setup、抗量子吗？（→ 轴二 PCS）
3. 非线性占比多少、表多大？（→ 轴三 查表）
4. 要不要流式/递归/聚合？（→ 轴四 折叠）

学完本讲，这四个问题对你不再是名词，而是带定量判断的决策。

---

1. 轴一：算术化——把计算变成代数的四种方言

所有证明系统的第一步都是同一件事：把"计算做对了"翻译成"某些多项式/约束满足"。翻译方式决定一切 downstream 成本。

方言	表示	单位代价	强项	弱项
R1CS	每门 $a \cdot z \circ b \cdot z = c \cdot z$	1 乘/门，但门数多（位运算	通用、最成熟（Groth16/PLONK 生	无查表原生支

方言	表示	单位代价	强项	弱项
	（三个线性组合一个乘）	爆炸）	态）、**Nova 折叠天然适配**（M05）	持；张量计算约束数爆炸
Plonkish	自定义门 + 查表门 + 拷贝约束	一个门可做任意 k 元一次组合	表达力最强（哈希一门化）、查表门原生	依赖多项式承诺的整表性质；设计门是手工活
AIR	执行 trace 的行间转移约束 `w(t+1) = F(w(t))`	每行一次转移	VM/递归类循环计算天然贴合（STARK 系）	非循环、数据依赖分支的计算表达别扭
sumcheck-native（GKR 式）	计算 = 分层/求和的多项式恒等式，claim 链归约	每层一次 sumcheck，**无电路编码**	张量线性代数（本系统 LLM 的形状）、无需承诺中间层	每算子要定制协议；分支/控制流弱

DeepProve 的坐标：sumcheck-native。**为什么对 LLM 是最优**：LLM 的 95% 计算是大张量矩阵乘——einsum sumcheck 直接证 `Σ_k A·B`，约束数量与张量元素同阶而非同阶×常数倍电路膨胀；非线性用查表旁路（轴三）。**若把 GPT-2 写成 R1CS**（zkML 的做法），一个 32-bit 乘法要拆几十个门——论文 §6 判 zkML "reduced efficiency and massive architecture requirements" 的机制就在这。

专家级追问（作业会考）：sumcheck-native 对**分支/循环**弱——如果负载是"证一个 RISC-V 程序跑了 N 步"呢？答案指向 AIR（zkVM 路线）——这就是轴一上"负载决定方言"的第一个实例。

2. 轴二：多项式承诺——三大族

承诺方案回答"表没被调包"（S07）。工程上只有三大族，各有死穴：

族	代表	证明大小	验证	setup	抗量子	一句话定位
配对族	KZG → Gemini → **HyperKZG**	O(1)，最小	O(1) 次配对（贵但常数）	**要**（毒物仪式）	✗	验证方最小的选择；聚合证明的主力
哈希族	**FRI/STARK → BaseFold → WHIR → Binius**	对数级，较大	哈希重算（快，可并行）	**透明**	√	无仪式 + 小域高速算术 + 硬件友好；新一代系统的主流
内积族	Bulletproofs / IPA	对数级	O(n) 线性（或 log 变体牺牲证明）	透明	√	验证方算力极弱、证明可接受偏大时

多线性专属变体（DeepProve 需要 MLE 承诺）：**HyperKZG**（Gemini 变换把多线性点折叠成一元打开——S08 §4 你手验过）、**BaseFold**（FRI 折叠与多线性折叠合流——M02 手算）、**Zeromorph**（把 MLE 映成单变量 KZG）、**Binius**（二进制域塔上的哈希承诺）、**Dory**（内积型配对）。

DeepProve 的坐标：双 PCS 并存（论文 §5）——BaseFold（透明版）与 HyperKZG（小证明版），**PIOP 完全共享**。实验结论（表 1/2，你已经会读）：HyperKZG 证明小一半（11.4 vs 24.3 MB）但长序列证明更慢；BaseFold 透明但大。**这正是轴二的取舍在真实系统里的样子：没有全面赢家，只有工作负载匹配**。

3. 轴三：查表论证的谱系

方案	核心想法	代价特征	一句话
Plookup (2020)	查询与表**排序后逐位对齐**，商式证明相邻差为零	首个高效方案；排序代价大；单表	开山；被 LogUp 的多重度法取代为主流
Caulk / cq / Baloo (2022-23)	预先**承诺表本身**，查询=子向量证明	表承诺一次性、查询便宜	表巨大且固定时好；子表技术后来喂给了折叠
LogUp / LogUp-GKR	对数导数分数和 + 多重度 + GKR 折	无除法、多列天然、多实例批量	**DeepProve 的选择**（S09 你手跑过）

方案	核心想法	代价特征	一句话
(2023)	叠		
Lasso + Jolt (2023-24)	"现成查表"：不承诺大表，用小承诺组合出任意大表	小域（如 Goldilocks）+ R1CSVM 上极快	zkVM 时代的主角；与 LogUp 是互补不是替代

选型直觉：表是"每层动态小表"→ LogUp-GKR（DeepProve：GELU/requant 表随层变化）；表是"巨大静态统一表"（如 CPU 指令语义）→ Lasso 系；需要最小证明且表可排序 → Plookup 一脉。

4. 轴四：折叠与递归

- **递归（recursion）**：把"验证一个证明"做成另一个证明系统的算术化——昂贵（要内嵌验证器）；
- **折叠（folding）**：**Nova**（2021）的洞察——不证"验证器"，只把两次执行的 R1CS 项**累积**成一个 relaxed 实例，用一次小 MSM/哈希折叠；IVC（增量可验证计算）由此实用化（SuperNova 多电路、ProtoStar/HyperNova 扩展到高次/多折叠）；
- **与 sumcheck 的合流**：折叠思想已渗入 PCS（BaseFold 的折叠=FRI 的折叠=你 S08 手算的多线性折叠——**三个"折叠"是同一操作在不同身份下的样子**，这是 M02 的主题）；
- **DeepProve 的坐标**：没用折叠——chunk 切分+seed 链是**并行**方案而非累积方案；对"单次推理证明"这是对的；若要证"流式持续服务 10^6 次推理"，Nova 一族立刻变成候选（M05 的作业）。

5. DeepProve 在地图上的完整坐标（及边界）

```
算术化: sumcheck-native（einsum + 查表旁路）      ← 对 LLM 最优
PCS:    HyperKZG + BaseFold 双支持                  ← 取舍留给了部署方
查表:   LogUp-GKR（多列、多实例批量）              ← 对"每层动态小表"最优
折叠:   无（chunk 并行 + seed 链）                  ← 单次推理够用
域:     BN254 Fr（254-bit，配对友好）               ← 为 HyperKZG 服务
```

它会在哪输（地图边界的自觉）：

1. **要求免 setup** → BaseFold 版可用但证明大 2 倍；更彻底的答案是 M02 的 WHIR/Binius 一代；
2. **证 VM 程序**（循环/分支/内存）→ AIR + Lasso 系（zkVM 路线）的天下，sumcheck-native 的算子定制优势消失；
3. **验证方在浏览器/合约里且带宽金贵** → KZG 的 O(1) 证明无可替代（但 setup 意识形态冲突）；
4. **量子敌手** → 配对族出局；
5. **模型 >3B** → 内存天花板（论文自认，S11 §5）——需要空间高效 PCS 或折叠流式化。

6. 决策树（把地图变成肌肉记忆）

```
负载是什么？
├─ 大张量线性代数 + 少量非线性（LLM/推荐/风控模型）
│    → sumcheck-native + LogUp-GKR + 多线性 PCS（本系统）
├─ VM/指令序列（zkVM、證明程序执行）
│    → AIR/Plonkish + Lasso 系查表 + 小域哈希 PCS
├─ 聚合大量既有证明 / 持续流式计算
│    → 折叠（Nova 族）或递归聚合
└─ 验证方极弱（浏览器/链上）
     → 配对族 PCS 优先，接受 setup
约束覆盖检查：免 setup？抗量子？内存上限？→ 逐条过滤上面的选择
```

7. 动手作业

1. 给四个负载做选型并辩护（每条选择引用地图坐标 + 一个定量理由）：①证 Gemma-3 270M 的推理（手机验证）；②证 RISC-V 程序 100 万步；③免 setup 地证 5000 次 CNN 推理；④证一个 3B 模型在 24GB 显存单机的推理。
2. 把 WHIR、Binius、Zeromorph、Dory 四个点**写进轴二地图**（各一行：机制一句话、代价坐标、何时选它）——这是 M02 的预告作业。
3. 反方练习：假设导师主张"sumcheck-native 是 LLM 的唯一正解"，找出两个反例负载并论证。
4. 阅读对照：论文 §6 的三段竞品判词，逐句映射到本讲地图坐标（哪个轴、哪个点）。

做完放入 `homework/m01/`。过关标准：导师扮演恶意评审，对你的选型连问三层"为什么不是 X"，每层都有定量回答。

下一讲：M02——把轴二最大的空洞填上：FRI 与哈希承诺家族。你将**在 mod-13 钟里手跑一个 mini-FRI**（含作弊者的逃逸概率计算），然后明白 BaseFold/WHIR/Binius 为什么统治了新一代系统。

M02 · FRI 与哈希承诺家族

> S07 的承诺有一根刺：**trusted setup**。仪式里那个销毁的 τ，要求你相信"至少一个人是诚实的"；配对还不是抗量子的。新一代系统（STARK/BaseFold/WHIR/Binius）全部押注在另一条路：**哈希承诺**——本讲你先弄懂它的发动机 **FRI**，并且**在 mod-13 钟里亲手跑一个 mini-FRI**——包括一个会让你愣住的实验：一个偏离 1/4 的作弊者，在这个玩具规模下竟然能全身而退。看清"为什么玩具会输"，你就真正理解了"为什么真 FRI 要那么设计"。 前置：S02（钟表）、S03/S08（两种表身份、折叠）、S07（承诺要解决什么）。

---

1. 为什么需要第二条路

回顾 S07 的承诺 $C = \Sigma\, f_i \cdot W_i$：力量表 $W_i = \tau^i \cdot G$ 来自一个**知道 τ 的仪式**。这带来三个包袱：

1. **信任假设**：仪式参与者至少一人诚实（"可信 setup"）；
2. **不抗量子**：配对/离散对数在量子计算下瓦解；
3. **域被绑架**：必须用配对友好的大素数域（BN254），享受不到小域（64-bit 级）的算术速度。

哈希路线的回答：**不用任何魔法对象，只靠"哈希 + 抽查"**。代价：证明更大、验证做哈希重算。它成立的核心是一个漂亮的代数事实——**"次数低" 在数据上表现为"聚集"**。

2. 发动机：Reed–Solomon 码视角

把多项式 f（次数 ≤ d）在域 $D = \{d_0, \ldots, d_{N-1}\}$ 上的取值表看成一个**码字**：全体"次数 ≤ d 的表"构成 N 维空间里的一个**低维子集**（只有 (d+1) 个自由度）。关键性质：

> **任意一张表，要么本身是低次多项式的表，要么与所有低次表的"距离"都很远**（至少 $1 - (d+1)/N$ 的位置对不上）。

于是"证明次数低"可以改成"**证明这张表靠近码空间**"——而后者可以用**随机抽查**做：抽查一个位置抓不到（作弊者只改少量位置），但**抽查十几个位置全部命中的概率会指数塌掉**。FRI 的全部艺术在于：**不用读完表，就能验证"整张表离码空间很近"**。

3. FRI 机制：折叠 + 抽查

条件：域大小 N = 2^m，且 D 对"平方"封闭、对"取负"封闭（可取乘法子群）。协议两阶段：

- **折叠阶段**（m 轮）：P 发当前表的 Merkle 根；V 掷随机数 α_i；P 把表折半——对每个 $y \in D^2$（原域的平方像）：

  ```
  f_{i+1}(y) = (1−αᵢ)/2 · [f_i(x) + f_i(−x)]  +  αᵢ/2 · [f_i(x) − f_i(−x)] / x      (x² = y)
  ```

  直觉：把 f 分解为偶部 $A(X^2)$ + 奇部 $X \cdot B(X^2)$——**次数每次减半**（f_1 的次数 ≤ d/2）。α 的随机性保证：**如果 f 远离低次，那 f_1 对绝大多数 α 也远离低次**（proximity gap，定量见文献 [BCIKS20]——本课标注边界，不深推）。

- **抽查阶段**：V 随机抽若干 x，要求 P 出示 f、f_1、f_2、… 沿链的对应值 + Merkle 路径，核对每一环的折叠等式。任何一环对不上即拒绝。

最终 P 交出**剩下的常数**（次数 < 1），V 核对它与最后一轮一致。

4. mod-13 玩具：全程手算

钟表世界 mod 13。取 4 阶乘法子群 **H = {1, 5, 8, 12}**（生成元 8：8^0=1, 8^1=8, 8^2=64≡12, 8^3=5；对平方封闭：H^2 = {1, 12} √；对取负封闭：−1≡12、−5≡8 √）。

被证对象：表 = f(X) = **3X + 2** 在 H 上的取值（次数 ≤ 1）：

```
域序 (1, 5, 8, 12) → 表 [5, 4, 0, 12]        (f(5)=17≡4, f(8)=26≡0, f(12)=38≡12)
```

V 掷挑战 **α = 2**。折叠（对每个 $x \in H$，$x^2 \in \{1,12\}$；需要的逆元：1/2=7, 1/10=4, 1/3=9, 1/11=6）：

```
x=1:  偶部 (f(1)+f(−1))/2 = (5+12)/2 = 17×7 = 119 ≡ 2
      奇部 (f(1)−f(−1))/(2×1) = (5−12)/2 = (−7)×7 = 42 ≡ 3
      f₁(1²=1)  = 2 + 2×3 = 8
x=5:  偶部 (4+0)/2 = 2        奇部 (4−0)/10 = 4×4 = 16 ≡ 3
      f₁(5²=12) = 2 + 2×3 = 8
x=8 = −5:  与 x=5 平方同点 → 同样给 f₁(12) = 8 ✓(自洽)
x=12 = −1: 与 x=1 同点 → f₁(1) = 8 ✓
```

诚实者的折叠表：f_1 = [8, 8]（域 {1,12} 上）——**必然是常数**：f(X) = 3X+2 的偶部 A ≡ 2、奇部 B ≡ 3（都是常数），折叠后 $f_1(Y) = A + \alpha \cdot B = 2 + 2\times3 =$ **8，与 Y 无关** √。这就是"次数减半"在玩具上的样子：1 次 → 0 次。

再折一次：f_1 是常数 → 偶部 8、奇部 0 → $f_2 = 8 + \alpha_2 \times 0 =$ **8**（α_2 是 V 第二轮掷的）。P 交出最终常数 **8**，V 核对一致。诚实者全程通过 √。

5. 亲眼看着玩具被攻破（本讲最重要的实验）

作弊表：**g = [5, 4, 0, 0]**——它在前 3 个点等于 3X+2 的真值、只在 x=12 处把 12 改成 0。它与"次数 ≤ 1"码的**距离 = 1/4**。

折叠（同样 α=2）：

```
x=1:  偶部 (5+0)/2 = 9        奇部 (5−0)/2 = 9        → g₁(1) = 9 + 2×9 = 27
≡ 1
x=12: 偶部 (0+5)/2 = 9        奇部 (0−5)/24 = (−5)×6 = 9  → g₁(1) = 27 ≡ 1 ✓
自洽
x=5:  偶部 (4+0)/2 = 2        奇部 (4−0)/10 = 3        → g₁(12) = 8
x=8:  同点自洽 ✓
作弊者折叠表：g₁ = [1, 8](域 {1,12})
```

注意：四条折叠等式全部自洽——作弊者可以交出 g_1 = [1,8] 并通过第一轮全部抽查（x=1 与 x=12 两个视角强迫同一个 $g_1(1)$=1，是这条偏离路径的巧合，也是玩具域太小的症状）。第二轮：$g_2(1) = (1+8)/2 + \alpha_2 \times (1-8)/2 = 11 + 3\alpha_2$——作弊者交出这个值，**依然自洽**。

愣住之后想清楚三件事（这就是真 FRI 与玩具的全部差距）：

1. **域太小**：|D| = 4 时，"距离码 1/4"的表只有 1 个坏点，而折叠把坏点与好点配对搅匀——4 条等式对 2 个未知数，自由度刚好够作弊。真 FRI 要求域远大于次数（安

全参数级），此时"偏离 δ 的表"折叠后**对绝大多数 α 仍偏离 δ'**（proximity gap [BCIKS20]）——距离不消失；
2. **抽查次数**：真 FRI 每轮抽查 t 个位置，抓到概率 ≥ 偏离占比；多轮、多点连乘 → 逃逸 (1−δ)^t 指数塌缩。玩具 4 个点抽 1 个，抓到率只有 1/4；
3. **最终检查的实质**：真 FRI 的最后常数必须与**所有抽查链**一致且次数递减到底——作弊者要维持整条链，等价于让一张 δ-远的表在每轮折叠后仍然"看起来低次"——proximity gap 说这几乎不可能。玩具里它可能，因为一切数字都太小。

教学结论：mini-FRI 让你看清**机制**（折叠/抽查/一致性链）；**健全性的定量保证只在"域大小 ≫ 次数 × 安全参数"时成立**。这不是玩具的失败，而是它最重要的教学内容——大牛与新手之分，正在于知道每个机制在什么规模下才兑现它的承诺。

6. 从 FRI 到 BaseFold / WHIR / Binius（地图轴二的哈希族）

- **BaseFold**：把 FRI 的"域折叠"与**多线性折叠**（S08 §4 你手算过的那种）合流——折叠系数表的同时产生 FRI 层，承诺与打开一次完成。DeepProve 的透明 PCS 就是它（论文表 1/2 的 BaseFold 列）；
- **WHIR**（2024）：折叠 + 内嵌 sumcheck 的迭代版，当前哈希族多线性承诺的精度与效率标杆之一；
- **Binius**（2024）：直接在二**进制域塔** $F_2 \subset F_4 \subset \dots$ 上工作——哈希和位运算免费、小域紧凑，专配"位运算重"的负载（哈希电路、VM）；
- **Circle STARK / M31**：用 Mersenne 素数 $2^{31}-1$ 与圆群替代乘法子群——CPU 友好的 31-bit 算术，Circle STARKs (2024) 解决了"域里没有 2^m 阶乘法子群"时的折叠问题。

三族硬对比（决策表，配 M01 的地图）：

维度	KZG/HyperKZG	FRI/BaseFold/WHIR	IPA/Bulletproofs
setup	毒物仪式	**无**	无
证明大小	O(1)，最小	O(log n)，较大	O(log n)，中
验证	常数次配对（单次贵）	哈希重算（可并行）	O(n) 线性（慢）
抗量子	✗	√	√（基于离散对数假设的变体需注意）

维度	KZG/HyperKZG	FRI/BaseFold/WHIR	IPA/Bulletproofs
硬件友好	MSM 专用	哈希流水线，GPU/ASIC 极佳	通用
域选择	被配对绑架	**自由（小域红利）**	通用

DeepProve 的双 PCS 实验（论文表 1/2 的机制解释）：HyperKZG 证明小一半（11.4 vs 24.3 MB）但长序列证明更慢（对 witness 承诺的开销随长度上升）；BaseFold 透明、大。**同一套 PIOP 跑两种 PCS** ——这正是 M01 §2"PIOP 与 PCS 解耦"的实盘验证。

7. 小域的工程现实（M03 的预告）

哈希族解锁了 64-bit/31-bit/二进制小域——算术快 3~10 倍、哈希免费。但小域带来三个新问题（M03 展开）：位宽不够表达计算 → **扩展域**；Montgomery/延迟约减的工程形态；FFT 的 2-adicity 要求（Goldilocks 2-adicity = 32、M31 用圆群、Binius 用塔）。

8. 动手作业

1. 换 α = 3 重跑 §4 的诚实折叠（应仍得 $f_1 = [8,8]$——为什么必然？用 A + αB 解释）。
2. 构造一个偏离 **2/4** 的作弊表（改两个点），跑第一轮折叠，证明**没有任何 g_1 能同时满足四条一致性等式**（两条等式强迫 $g_1(1)$ 为两个不同的值）。由此体会"偏离占比 → 抓住概率"的门槛效应。
3. 把域换成 8 阶子群 H_8（生成元 $2^6=12$？——先验证 {1,12,…} 的阶），重做一次诚实折叠，体会 |D| 翻倍对作弊自由度的挤压。（选做）
4. Rust：实现 mini-FRI（4 点域、2 轮、Merkle 树用 blake3），跑诚实者与 §5 作弊者的完整流程，打印每一轮的表与抽查结果。
5. 把 BaseFold/WHIR/Binius/Zeromorph/Dory 五个点写进 M01 §2 的轴二地图（各一行）。

做完放入 `homework/m02/`。过关：导师随机指定域/挑战/作弊幅度，你现场重推。

下一讲：`M03 数域的选择学`——小域为什么快、位宽不够怎么办、以及"本系统为什么还留在 BN254"的完整账。

M03 · 数域的选择学：为什么新一代系统都在"换钟"

> S02 说"所有计算活在素数域上"，本仓库选了 254-bit 的 BN254 Fr。但 2023 年以来的新系统（Plonky3/Binius/Circle STARK/Jolt）几乎全在**小域**上跑。这一讲讲清这笔账：小域快多少、快在哪、**会坏在哪**（你会在 mod-13 玩具里亲眼看到"装不下"），以及两个修复方案——更大的钟与**扩展域**（你在 mod-13 里亲手造出 169 格的大钟，Binius 的塔就是这个构造的直系后代）。最后回答：DeepProve 为什么留在 BN254。 前置：S02（钟表）、S06（sumcheck 与域无关的原因）、M02（FRI 需要 2^m 阶子群）。

---

1. 域的大小直接写进乘法器的电路里

一次钟表乘法的硬件代价 ∝ 钟的位数：

域	位数	一次乘法的形态	量级感受（CPU，量级非精确 benchmark）
BN254 Fr	254 bit（4×64 limb）	Montgomery CIOS：十几次 64×64 乘加 + 约减	基准（~40–60 ns 量级）
Goldilocks `2^64−2^32+1`	64 bit（**单 limb**）	一条 64×64→128 乘 + 特殊模数的一次减法约减	快约一个量级
Mersenne31 `2^31−1`	31 bit	32-bit lane；AVX2/AVX-512 **一条指令并行 8~16 个乘法**	再快数倍，GPU/SIMD 的甜点
二进制塔（Binius）	1 bit 起步	加法 = XOR（免费），乘法 = 位逻辑	哈希类计算的天堂

这就是换域的动机：**同样的 sumcheck/查表机器，换到小域上整机快数倍**——新一代系统集体换钟的原因，一句话讲完。

2. 小域的第一个坑：装不下（mod-13 玩具实验）

在 mod 13 里算 **5 × 5**：钟表给出 5×5 = 25 ≡ **12**。但整数世界的事实是 25。如果协议需要的是"25"这个**整数语义**（比如"格子的乘积应等于 25"），mod 13 直接给错——不是算错，是**世界太小，25 这个数不存在**。

真实对应：S09 讲过 b-bit × b-bit 的积要 2b 位（requant 的位长公式 q + log n + 1 就是它）。LLM 的中间量是 i64、乘积会到 128 bit——放进 31-bit 的 M31？**溢出 = wrap-around = 正确性论证直接失效**（域里 25 和 12 无法区分，作弊与诚实不可分辨）。

> **铁律**：域阶必须大于计算链条中**最大中间值的界**（含所有乘积与累加），否则一切证明失去意义。

BN254 Fr（~2^{254}）为什么安全：2^{128} 级的安全参数下，i64 乘 i64 的界（2^{128}）< 2^{254} √（这正是 S01 域化一致性论证的前提）。M31（2^{31}）显然装不下——**所以小域系统必须配套"表示学"**，见 §4–5。

3. 解法一：换一台刚好大的钟——Goldilocks

$p = 2^{64} - 2^{32} + 1$，自证它的约减魔法（一行）：p ≡ 0 意味着 **$2^{64} \equiv 2^{32} - 1 \pmod p$**——任何 128-bit 乘积的高位可以一**次减法**折回低位。

- 单 limb：乘法就是原生 `uint64` 运算；
- **2-adicity = 32**：存在 2^{32} 阶乘法子群 → 能做 2^{32} 点的 FFT/FRI（M02 折叠树的域）；
- 剩一个坑：32-bit × 32-bit 的积（≈2^{64}）可能**恰好超过 p**（p ≈ 1.83×10^{19}，2^{64} ≈ 1.84×10^{19}）——所以 Goldilocks 系统对乘积要做两段拆分（这是它生态里反复出现的"lazy reduction"约定）。

4. 解法二：扩展域——在 mod 13 里亲手造一台 169 格的钟

小域装不下，可以**造更大的钟**。在 F_13 上找一个**不可约**二次多项式：试 $X^2 - 2$。2 是不是 mod 13 的平方？查 $2^6 = 64 \equiv 12 \equiv -1$（非平方的标志）→ **$X^2 - 2$ 无根、不可约** √。于是造出：

```
F₁₆₉ = F₁₃[α]，其中 α² = 2      （元素形如 a + bα，共 13² = 169 个）
乘法：(a+bα)(c+dα) = (ac + 2bd) + (ad + bc)α
例：(1+2α)(3+α) = (3 + 2×2×1) + (1×1 + 2×3)α = 7 + 7α      （自己展开验算：
3+α+6α+2α² = 3+7α+4 ✓）
```

三个要点：

1. **装得下了**：169 > 100——如果计算需要区分"100 和 12"，F_{13} 做不到、F_{169} 做得到。真实世界：M31 装不下 64-bit 值 → 用 **$M31^2$**（62-bit 级）；F_2 装不下任何东西 → **塔** $F_2 \subset F_4 \subset F_{16} \subset \dots \subset F_{2^{128}}$（Binius 的"塔"就是这个：每层都是上一层的二次扩展，嵌套算术免费）；
2. **扩展域的元素仍是多项式**——sumcheck/查表协议原封不动地在扩展域上跑（S06/S09 的机器不挑域）；
3. **安全性的角色**：FRI 的健全性需要域 ≫ 次数（M02 §5 的教训），小域系统用**低度扩展域**来满足它——扩展域是"装值"和"保安全"的双重解法。

5. 2-adicity：FFT/FRI 的入场券（回到你自己的玩具）

S02 说 $12 = 2^2 \times 3$ → **mod 13 的 2-adicity = 2**，乘法群里最大 2 幂阶子群是 4 阶——**正是你在 M02 手跑 FRI 用的 H = {1,5,8,12}**，折两层恰好用尽。2-adicity 决定 FFT/FRI 的最大规模：

域	2-adicity	最大 FRI/FFT 规模
mod 13	2	4 点（你已用完）
Goldilocks	32	2^{32} 点
BN254 Fr	28	2^{28} 点
M31	**1**（$2^{31}-2 = 2\times(2^{30}-1)$）	2 点！→ Circle STARK 用"圆群"绕开（一句话级）

6. DeepProve 为什么留在 BN254（完整账）

1. **HyperKZG 绑定配对** → 绑定 BN254 这条配对友好曲线；BaseFold 版也在 BN254（Ceno 库实现）；
2. **换域的理论成本极低**：sumcheck/LogUp 是"域无关"的（只要求"一个域"）——换域只需换 PCS 与曲线，PIOP 原封不动。**S10 讲的 PIOP/PCS 解耦在这里第二次兑现**：论文的 BaseFold 版就是同一套算子协议换了个 PCS；
3. **不换的代价**：254-bit 算术比 Goldilocks 慢一个量级；查表/sumcheck 占 50%+ 的时间本可以再缩；
4. **不换的理由**：配对验证方极便宜（1 次配对）、生态成熟（arkworks 全家桶）、GPU MSM 已有（仓库带 cuda feature）；
5. **前沿方向**：迁 Goldilocks/31-bit + 哈希 PCS = "PIOP 不动、底盘全换"的改造——这是 M08 顶石设计的候选题之一。

7. 选型三问（把方法装进口袋）

1. **值域上界多大?** （含乘积与累加 → 域阶铁律；不够 → 扩展域或换大域）
2. **承诺走哪族?** （配对 → 大素数域；哈希 → 小域+2-adicity；混用 → 双域系统）
3. **硬件目标?** （CPU 大核 → 64-bit；SIMD/GPU → 31-bit；FPGA/ASIC 位运算 → 二进制塔）

三个负载练习见作业。**方法总结**：域不是信仰，是"值域 × 承诺族 × 硬件"三约束的联立解。

8. 动手作业

1. 在 F_{169} 里算 $(2+\alpha)^2$ 并展开（答案：$4 + 4\alpha + \alpha^2 = 4+4\alpha+2 = 6+4\alpha$）；再算它的平方根不存在性——即方程 $(a+b\alpha)^2 = 1+0\alpha$ 是否有非平凡解（提示：比较分量得 $ac+2bd = 1$、$ad+bc = 0$ 的方程组，讨论 $b=0$ 与 $b\neq0$）。
2. 证明 $X^2 - 2$ 在 F_{13} 不可约的另一种方法：列出 0..12 的平方表，确认没有平方 $\equiv 2$。
3. 验证 Goldilocks 的 2-adicity：$p-1 = 2^{64}-2^{32} = 2^{32}(2^{32}-1)$，而 $2^{32}-1$ 为奇数 → 恰好 2^{32}。

4. LLM 中间量是 i64（$|x| < 2^{63}$）。回答：直接进 M31？进 $M31^2$（62-bit）？进 Goldilocks？各有什么坑？（提示：负数、乘积 2^{64} 位、requant 的除法语义。）
5. 设计题（M08 预演）：若把 DeepProve 迁到 Goldilocks + FRI 系 PCS，列出演义级改动清单（哪些组件零改动、哪些重写、哪些新增），并估-main-bottleneck 的变化方向。

做完放入 `homework/m03/`。

下一讲：`M04 查表论证全景`——从 Plookup 到 Lasso/Jolt 的完整谱系，用 S09 的玩具表验证"差分劈开"恒等式，并回答：为什么 DeepProve 选 LogUp-GKR 而不是它们。

M04 · 查表论证全景：从 Plookup 到 Lasso/Jolt

> S09 教了 LogUp——但它是谱系上的第三/四代。本讲把整条谱系摊开：**每一代都在削减"必须承诺的东西"**——Plookup 要排序并承诺整张表，Caulk 把表承诺挪到一次性，Lasso/Jolt 干脆**不承诺大表**。这是 S10"虚拟主张"原则（验证方能算的/能省的都不承诺）在查表方向上四年的演进史。你会在 S09 的玩具表上亲手验证 Plookup 的"差分劈开"恒等式，并最后回答：DeepProve 为什么选 LogUp-GKR。
> 前置：S09（LogUp 全程）、M01（地图轴三）、M02（域与承诺的取舍）。

---

1. 谱系主线（先给结论）

```
2020  Plookup          证明"查询在表里"：排序合并 + 差分劈开          （要排序、承诺整表）
2022  Caulk/cq/Baloo   表预先承诺，查询 = 子向量证明                  （去排序；表承诺挪到一次性）
2023  LogUp            多重度 + 对数导数分数和                        （去排序；多列/多集自然；S09）
2023  LogUp-GKR        分数折叠树 + GKR 分层                          （可批量；DeepProve 选择）
2023  Lasso            分解式查表：大表 = 小承诺表的组合              （不承诺大表！）
2024  Jolt             把 RISC-V 每条指令的语义做成 Lasso 查表        （zkVM 提速器）
```

一句话总结四年的方向：**承诺的清单越来越短**。

2. Plookup 的核心直觉：排序合并 + 差分劈开（玩具可验）

想法：把"查询 ⊆ 表"翻译成"表与查询的**排序合并序列**的相邻差，恰好把表的相邻差**劈开**"。

玩具（S09 同款表）：T = [1, 2, 3, 4]（mod 13），查询去重后 F = {1, 3}。合并排序：

```
s = [1, 1, 2, 3, 3, 4]        (f ∪ F 的多重集排序：查询 1 插在两个 1 之间，3 同理)
```

差分劈开恒等式：表的环形差 = {1, 1, 1, −3}（即 1→2, 2→3, 3→4, 4→1 的 −3）；s 的环形差 = {0, 1, 1, 0, 1, −3}。逐一核对"表的每个差 = s 中落在该区间内的差之和"：

```
+1(1→2) = 0(1→1) + 1(1→2)   ✓      查询 1 把表差劈成了两半
+1(2→3) = 1(2→3)             ✓      无查询插入
+1(3→4) = 0(3→3) + 1(3→4)   ✓      查询 3 劈开
−3(4→1) = −3                  ✓      wrap 段无插入
```

为什么这条恒等式 ⟺ 查询合法：只有当每个查询值都落在表里（且次数被正确记录）时，合并序列才恰好把表差劈成"和不变"的小段；多查一个表外的值会引入一个不属于任何表区间的差。验证方式：V 掷随机挑战，把两侧的差分序列用**随机权重的乘积式聚合**压成少数几个多项式检查（Plookup 原文用相邻差的线性形乘积；后来 LogUp 把这一步换成了对数导数分数——同族思想的两代实现，LogUp 那代免排序且天然支持多列）。

诚实边界：Plookup 原文还有两块本讲不复现的机制——①"域上没有大小序"，"排序"本身要用 0/1 标志位 + 额外约束去证明；②完整的商式检查（原论文 §2.1）。这正是它复杂度的来源，也是下一代要干掉的东西。深挖任务见作业 3。

3. Caulk / cq / Baloo（2022–23）：把表承诺挪到"一次性"

Plookup 每次证明都要把**整张表的多项式**卷进协议——表 6.5 万格就得每次付 6.5 万格的代价。Caulk 的改法：**表在 setup 期承诺一次**；每次证明只需证明"查询值对应表的一个**子向量**"——用表承诺的"部分打开"完成。cq/Baloo 把它推广到多查询、任意子表。

代价转移：查询侧变便宜了，但要求**表是静态的**（承诺一次反复用）——对"每层都在换表的 LLM"不友好，对"一张巨大固定表"（如椭圆曲线标量乘的位分解表）正好。子向量打开的思想后来被折叠族吸收（M05）。

4. Lasso + Jolt（2023–24）：连表都不承诺

谱系的激进一代。Lasso 的洞察：一张大表 T: [m] → F 可以分解成**若干张"数字表"的线性组合**——承诺 K 张长度 log(m) 的小表，就能虚拟出长度 m 的大表，承诺开销从 O(m) 降到 O(K·log m)。**"现成的"（off-the-rack）查表**：表不用预先承诺，查询时按需组装。

Jolt 把这个武器对准了 zkVM 的最大痛点：RISC-V 每条指令的语义（加法/移位/符号扩展…）都做成小查表的组合——程序执行的正确性 = 逐指令查表正确性的总和，VM 的"SNARK-unfriendly"部分被查表吃掉。这使 Lasso/Jolt 在小域（Goldilocks/M31）+ 哈希承诺的生态里成为 zkVM 的事实标配。

与 LogUp 的关系：不是替代，是**适用域互补**——Lasso 的分解要求表有"数字分解"结构（位宽驱动的表），LogUp 要求"多重度记账"结构。两者都是"少承诺"哲学的孩子。

5. 四代对比总表（决策用）

维度	Plookup	Caulk/cq	LogUp-GKR	Lasso/Jolt
排序	**要**（域上无序，需额外机制）	不要	**不要**	不要
表承诺	整表、每次	整表、一次性	setup 表 + witness（S08 收口）	**不承诺大表**
多列（位切片）	别扭	单列为主	**自然**（S09 §4 的 α_col 合并）	经数字分解支持
多实例批量	弱	弱	**天然**（GKR 折叠树，S09 §5）	天然
证明大小	O(1) 配 KZG	O(1)	O(log·层数)	O(log·K)
甜点负载	小表 + 最小证明	巨大静态表	**每层动态小表 + 位宽切片**	VM 指令语义、位运算重

维度	Plookup	Caulk/cq	LogUp-GKR	Lasso/Jolt
代表系统	早期 Plonkish	专用查表链	**DeepProve**	Jolt-RISC-V

6. DeepProve 的选择账（为什么是 LogUp-GKR）

回到它的负载：**每个查表层的表是动态小表**（位宽 16 → 6.5 万格 × 若干位切片列），**每层多实例**（序列里每个位置都要查），**域是 BN254**：

- 排序？6.5 万格 × 每层每实例 → 排序代价不可接受 → 排除 Plookup；
- 表静态？**不**——witness 里的查询/输出每层都变 → 排除 Caulk 系的"一次性承诺"红利；
- 多列 + 多实例批量？LogUp-GKR 的折叠树天然吃下（α_col 合并列、多实例并轮）——**正中靶心**；
- 证明大小？LogUp-GKR 产 log 级证明，收口后并入 batched sumcheck（S08）——与系统其它部分同构。

对称的边界（M01 的纪律）：如果负载换成"一张 2^{32} 的固定指令语义表、百万条指令"——LogUp-GKR 的多重度承诺会爆，Lasso/Jolt 碾压。**没有最好的查表，只有配得对的查表**。

7. 动手作业

1. 差分劈开换表验证：T = [1, 2, 3, 4]、查询 F = {3, 1}。合并排序 s = [1, 1, 2, 3, 3, 4]，验证四段表差（1, 1, 1, −3）各自被 s 的差劈开且和不变。（结构与本讲 §2 完全同构，锚点：s 的环形差 = {0, 1, 1, 0, 1, −3}。）
2. 递减表会怎样？T = [4, 3, 2, 1]、F = {1}：合并 s = [1, 1, 2, 3, 4]，环形差 s 侧 = {0, +1, +1, +1, −3}，而表差 = {−1, −1, −1, +3}——**符号全部对不上**。写出劈开恒等式需要的**方向/符号处理**，并解释为什么这暴露了"排序合并隐含依赖域上的大小序约定"的隐藏复杂度（这是 Plookup 工程代价的具象化；锚点数字已核验如上）。
3. 填空对比表：把 §5 的表抄一遍，凭记忆填 7 个维度，再对照检查；对填错的维度回到对应小节重读。
4. （M07 预演）读 Lasso 论文 §1，写一张 claim 卡片：问题 / 核心引理 / 承诺开销公式 / 与 LogUp 的适用域边界——格式照 S11 §0 的四件套。

5. 思考：DeepProve 的 requant 表（位分解 + InRange + Comb 三张表，S11 §4.3.2）如果换到 Lasso 框架下，哪些表能被"数字分解"吸收、哪些必须保留 witness 承诺？（开放题，写入 homework 供讨论。）

做完放入 `homework/m04/`。

下一讲：`M05 折叠与递归`——Nova 的 relaxed R1CS 一步手推，folding 与 sumcheck 折叠的合流，以及"给流式证明长视频设计折叠方案"的顶石练习。

M05 · 折叠与递归：Nova 一族

> 问题：证明"一个跑了 **N 步**的计算"。两条傻路：①把 N 步全部塞进一个电路（N 步 = N 倍门数，proof 系统被撑爆）；②每步证一次、验证方读 N 个证明（违背"短"，S01）。正统的第三条路——**递归**（把"验证上一个证明"做成电路）——又贵得惊人（配对验证器进电路要百万门）。**Nova**（2021）给出了第四条路：**不证旧证明，把旧实例"折叠"进新实例**——一次小线性运算。本讲你在 mod-13 里亲手完成一次 Nova 折叠，并验证它对**任意**挑战成立。 前置：S06（约束思想的雏形）、S07/S08（随机挑战/承诺）、M01 §4（地图轴四）。

---

1. 先把"实例"写成代数形状

R1CS（M01 轴一的第一方言）把每步计算写成一组约束 `Az ∘ Bz = Cz`（∘ = 逐元素乘；A、B、C 是公开矩阵，z 是"赋值向量" = 公共输入 + witness）。Genuine（诚实）的 z 满足等式。

Nova 的关键改写——**relaxed R1CS**（"放松的"）：

```
Az ∘ Bz  =  u·Cz + E
```

多了一个标量 u 和一个**误差向量 E**。诚实实例是它的特例：u = 1、E = 0。**E 度量"这个实例离真有多远"**——记住这一点，折叠的全部魔法都建立在它上面。

2. 玩具：一条约束的 R1CS（mod 13）

最小系统：A = [1]、B = [1]、C = [2]，z 是单个数。约束：**z·z = 2z**（即 z ∈ {0, 2}）。

```
genuine 实例：z₂ = 2（2×2 = 2×2 ✓），u₂ = 1，E₂ = [0]
累积器（relaxed）：z₁ = 4，u₁ = 3，E₁ = [5]
   自检：Az₁∘Bz₁ = 4×4 = 16 ≡ 3；u₁·Cz₁ + E₁ = 3×(2×4) + 5 = 29 ≡ 3 ✓  (mod 13)
```

注意累积器 **不满足真约束**（4×4 = 16 ≠ 2×4 = 8）——它"欠着"一个误差 E，但这个欠账被**代数地记录在案**。这就是"relaxed"的含义：不要求每步都完美，要求**误差可控、可追踪**。

3. 折叠：一次线性运算（核心公式）

把累积器 z_1 与一个**新鲜 genuine 实例** z_2 用随机挑战 r 折叠：

```
z* = z₁ + r·z₂        u* = u₁ + r·u₂        E* = E₁ + r·T
其中交叉项 T = Az₁∘Bz₂ + Az₂∘Bz₁ − u₁·Cz₂ − u₂·Cz₁
```

代入玩具（T 的四个量：$Az_1 = 4$、$Bz_2 = 2$、$Az_2 = 2$、$Bz_1 = 4$、$Cz_2 = 4$、$Cz_1 = 8$）：

```
T = 4×2 + 2×4 − 3×4 − 1×8 = 8 + 8 − 12 − 8 = −4 ≡ 9 (mod 13)
```

验证折叠正确（对任意 r）。折叠后要求：$z^2 = u \cdot C(z^*) + E^*$。左：

```
z*² = (4 + 2r)² = 16 + 16r + 4r² ≡ 3 + 3r + 4r²   (mod 13)
```

右：

```
u*·C(z*) = (3+r)·(2(4+2r)) = (3+r)(8+4r) = 24 + 12r + 8r + 4r² ≡ 11 + 7r + 4r²
+ E* = [5 + 9r]
合计 = 16 + 16r + 4r² ≡ 3 + 3r + 4r²    ✓✓ 与左边逐系数相等
```

代 r = 5 落地：z* = 14 ≡ 1，u* = 8，E* = [50 ≡ 11]；核：$z^2 = 1$，$uCz^* + E^* = 8 \times 2 + 11 = 27 \equiv 1$ √。

为什么恒等（两行推导，自己补全）：把 $(z_1+rz_2)^2$ 展开 = $z_1^2 + r(z_1z_2+z_2z_1) + r^2z_2^2$，其中交叉项 $z_1z_2+z_2z_1$ 恰好被 T 吸收——**T 就是"折叠时产生的交叉项"的精确记账**。$E^* = E_1 + rT$ 的含义：**旧误差原样保留 + 新交叉项记账**。诚实者折叠后的 E 依然小；不诚实的输入会把误差暴露在 E 里。

4. 为什么这就解决了"100 万步"

IVC（增量可验证计算）的新范式：

```
累积器 ←折叠← 新鲜实例₁ ←折叠← 新鲜实例₂ ←折叠← … ←折叠← 新鲜实例_N
```

- 每一步的真实计算（z_{i+1} = step(z_i)）与折叠的交叉项 T **一起**在一个小电路里证明（Nova 的"折叠电路"），挑战 r 由 Fiat–Shamir 从 S05 的 transcript 派生；
- N 步之后：**一个**累积器 + **一个**最终证明。验证方：验最终证明 + 抽查折叠电路链——不读 N 个证明；
- 你在 S06/S08 学的全部机器没有作废：Nova 里证明"折叠电路执行正确"仍然用 sumcheck/PCS——**折叠是外层的调度术，内层还是你熟悉的发动机**。（HyperNova 进一步把折叠推广到 sumcheck 原生的 CCS 关系——folding 与 sumcheck 在合流，这是当前协议设计的前沿方向。）

与 DeepProve chunking 的对比（M00 缺口诊断的直接回答）：

	chunking（DeepProve）	**折叠（Nova 族）**
结构	并行分治，chunk 间承诺绑定	串行累积，一步包一步
验证方	逐 chunk 核对（数量 ∝ chunk 数）	恒定（最终一个证明）
适用	单次推理、可并行、内存受限	长流式计算（IVC）、持续服务
证明延迟	每 chunk 独立	串行链（折叠必须逐步）

LLM 单次推理选 chunking 是对的；"证明服务运行了 10^6 次推理"就该轮到折叠——**同一张地图上的两个坐标，不是谁取代谁**。

5. 动手作业

1. **再折一步**：以 §3 的 (z* = 1, u* = 8, E* = [11]) 为累积器，再折叠一个新鲜实例 $z_3 = 2$（r 自选）：算 T′、E′，并验证 $z^{*\prime 2} = u^{*\prime}\cdot Cz^{*\prime} + E'$。（锚点：$T' = Az^{*}\circ Bz_3 +$

$Az_3 \circ Bz^* - u^* \cdot Cz_3 - u_3 \cdot Cz^* = 1\times2 + 2\times1 - 8\times4 - 1\times2 = 2+2-6-2 = -4 \equiv 9$；$E' = [11 + 5\times9] = [56 \equiv 4]$；$z^{*\prime} = 1+2r'$, $u^{*\prime} = 8+r'$……请把完整验证做完。）

2. **漂移实验**：把一个**非 genuine** 实例（$z_2 = 4$, $u_2 = 3$, $E_2 = [5]$）折叠进累积器。展开后你会发现多出 $r^2 \cdot E_2$ 项无处安放——写出这个"误差漂移"，并解释为什么 Nova 只允许"累积器 × 新鲜实例"的折叠方向。
3. 证明：若累积器满足 relaxed（u_1, E_1）且折叠的新鲜实例满足 genuine（$u_2 = 1$, $E_2 = 0$），则折叠结果满足 relaxed（$u^* = u_1 + r$, $E^* = E_1 + rT$）——把 §3 的两行推导写成全句子证明。
4. Rust：用 `ark-bn254::Fr` 实现 1-约束玩具的 fold 函数（20 行），随机 r 测试 100 次，全部满足 relaxed。
5. 设计题（M08 预演）：用折叠方案设计"证明一台服务处理了 10^6 次 LLM 推理请求"——画出累积器结构、每步电路要证什么、验证方的最终工作清单。对比 DeepProve chunking 方案的成本差异。

做完放入 `homework/m05/`。

下一讲：`M06 安全与攻击者视角`——健全性预算表、你亲手实施的第一个攻击（statement binding：如果 transcript 忘了吸收承诺，"证明任何表的总和 = 6"在 mod-13 里真实可行）、以及从 Frozen Heart 到 Binance 事故的审计清单。

M06 · 安全与攻击者视角：健全性预算、真实攻击与审计清单

> 前九讲都在建机器；这一讲教你**拆机器**。顶尖与熟练的真正分界在此：能建的人是工程师，能拆的人才是安全专家。你将：①给 DeepProve 做**健全性预算表**（把所有"逃逸概率"记成一页账）；②亲手实施第一个攻击（statement binding——在 mod-13 里真实可行）；③从 Frozen Heart 与 Binance 事故里提炼**审计清单**。 前置：S05（随机挑战）、S06（sumcheck 健全性）、S07（承诺）、S08（收口）、S10（系统全景）。

---

1. 健全性预算表：把"安全"变成一页会计账

协议的每个环节都贡献逃逸概率。大牛拿到任何系统，第一件事是列这张表（DeepProve 的口径）：

环节	逃逸概率	数量	小计
sumcheck 单轮（≤ d/2^{128}，d=6）	≈2^{-125}·1.5	每实例 n 轮	—
sumcheck 单实例（n=20 轮合计）	≈2^{-120}·1.9	每节点 1 实例	—
sumcheck 全部实例	≈2^{-120}·1.9	V ≈ 500（GPT-2 量级）	≈2^{-111}
PCS 打开（HyperKZG）	~2^{-128}	每 chunk 1 次	2^{-128}
batched sumcheck 的 γ 组合	~2^{-128}	每 chunk 1 次	2^{-128}
chunk 间 seed 链/边界承诺	绑定性归入 PCS	—	—
合计			**≈2^{-111}（由 sumcheck 实例数主导）**

读表的两条纪律：①逃逸概率相**加**（攻击者只需一条路），所以最弱的环节决定水位——没有"平均安全"；②每个数字要有出处（挑战熵 2^{128} 来自 S05 的 16 字节 XOF 口径；d 来自 S06 的轮多项式次数）。**给不出出处的安全数字 = 营销**。

2. 亲手实施第一个攻击：statement binding（陈述绑定）

场景：V 要验证"承诺为 C 的表，总和 = 6"。协议：P 跑 sumcheck（S06）→ 终点主张 f(7,4) = 11 → V 用 C 打开核销。

漏洞设定（导师给学生的坏协议）：transcript 里**忘记吸收 C**——挑战 r = (7,4) 只依赖 sumcheck 消息，不依赖"承诺的是哪张表"。

攻击：作弊者根本不用碰 sumcheck——它的消息是 f 的、原样重放。他只需**事后**提交一张在 (7,4) 处取值为 11 的新表。从 S08 知 eq((7,4),·) = [5,5,2,2]，所以：

```
g = [10, 0, 0, 0]   →   g(7,4) = 5×10 + 0 + 0 + 0 = 50 ≡ 11 ✓  (mod 13)
```

V 的全部检查：①sumcheck 消息自洽（与表无关，通过）；②终点主张 11 √；③打开 C_g 在 (7,4) 得 11 √——**全部通过**。V 被迫接受"g 的总和 = 6"，而 g 的真实总和是 10。**一张自选的表 + 重放的证明 = 伪证**。

修复 = 协议顺序：承诺必须**先于**挑战进 transcript（真实系统的顺序：承诺 → 吸收 C → 派生 r → sumcheck → 打开）。修好后 r 依赖 C，攻击者想找"g 在新 r 处 = 声称值"就要在 2^{-127} 概率下撞点——回到 S05 的老账。**你刚体验的攻击有一个正式名字：statement binding 缺失**；它的真实受害者见 §4。

再进一步（攻击者视角的深化）：即使吸收了 C，攻击者还可以**磨**（grinding）——反复换自己的新鲜表重跑协议，赌挑战恰好落进可乘之机，每次 $\approx 2^{-127}$。防御：让每次磨的成本足够高 + 熵足够大。**攻击者视角永远问：哪一步可以离线重试?**

3. 真实世界的事故（只列名字与教训，细节按名检索）

事故	教训	对应本课程
Frozen Heart 系列（Trail of Bits, 2022）	多个主流实现（Plonky、Dusk 的 Plonk 等）的 Fiat–Shamir transcript 缺陷：漏吸收、错吸收、挑战派生错误 → 完全可伪造	§2 的"顺序即协议"+ FPGA 文档附录 A 的标签表
Binance "Alice" 资产伪造（BSC, 2022）	旧证明在**新陈述**上重放（statement binding 缺失）	§2 你刚实施的攻击的工业版
Zcash 反超发漏洞（BCTV14，披露于 2018/19）	承诺方案内部碰撞 → 可以凭空铸币	承诺绑定性（S07）不是可选性质

模式：三类事故对应三种绑定缺失——绑挑战（FS）、绑陈述（replay）、绑承诺（碰撞）。**审计 = 逐一检查这三种绑定**。

4. 本仓库的攻击面清单（从源码读出，逐条已核）

1. transcript 吸收顺序/字节格式（附录 A 标签表；`read_challenges` 连续同值的实现现状——挑战熵按现状计）；

2. chunk seed 链的重放顺序 = 完成顺序（streaming 语义）；批式验证的公共挑战口径不同——两条路径不能混；
3. 边界承诺相等（相邻 chunk 对同一张量）——绑定的强度 = MSF；
4. **量化溢出不在证明范围内**（S02：域化一致性以 i64 不溢出为前提）——溢出检查是模型编译期的责任，审计时别漏掉这条"证明范围之外"；
5. 采样随机性（论文 §4.3.6：种子要用可验证随机数来源）；
6. `read_challenges`/`append_and_sample` 的任何一处新增吸收点必须两侧同步 + 登记附录 A。

5. 审计清单（可执行的 8 步，照单执行）

1. 画出 transcript 的**完整吸收序列**（逐字节），与附录 A 比对；
2. 找出**所有**随机挑战派生点，确认吸收先于派生、无遗漏、无重复；
3. 检查每个 challenge 是否被**绑定到具体陈述**（表/语句/公共输入）；
4. 列健全性预算表（§1 格式），标注每个数字的出处；
5. 检查"证明范围之外"的假设（溢出、采样、输入合法性）是否有编译期/应用层兜底；
6. 找离线可重试点 → 估算 grinding 成本；
7. 检查跨实例/跨 chunk 的重放面（旧证明能否在新陈述上使用）；
8. 用负例测试固化：每个"应拒绝"的构造写成回归测试（S06 P2 的纪律）。

6. 动手作业

1. **攻击实验**：按 §2 完整实施——先跑 S06 的诚实 sumcheck 得终点主张，再构造 g = [10,0,0,0] 重放，写出 V 的每步检查与结果；然后加上"C 先入 transcript"，重放并指出哪一步首次失败。
2. 预算表：对你 W8 跑过的 GPT-2 证明，按 §1 格式列 DeepProve 的完整预算（V 取实际节点数），计算合计安全水位。
3. 审计练习：把清单 §5 的 8 步对 `prove_chunk_详解` 的收口三步逐条过一遍，写一页"审计意见"（至少提出一条可改进项——真实答案存在，例如输出 evals 与边界承诺的核对顺序）。
4. 案例 grep：检索 "Frozen Heart vulnerability Trail of Bits"，挑一篇复盘：漏洞与 §2 的哪种绑定缺失同构?

做完放入 `homework/m06/`。

M07 预告：方法论模块——把 S11/S09 的精读产出变成"claim 卡片"，把卡片变成最小实现，把实现变成上游 PR。M-track 的四讲理论（M01–M04）+ 两讲实践基础（M05/M06）已齐，接下来是把它们投入真实战场。

M07 · 从论文到代码：实现方法论

> 本讲没有新理论——它是把 L4（会造）变成**可重复的手艺**的流程模板。四步：**claim 卡片 → 最小实现 → 三板斧验证 → 上游 PR**。每步给模板、给常见死法、给时间预算。本讲的任务就是流程本身：你的交付物是一个真实的上游 PR（L08 P11 的一种完成方式）。 前置：S11（精读能力）、S06–S09（协议库存）、L07 §2（dump 与对拍工具箱）。

---

1. Step 1 · Claim 卡片（读论文的产出物）

读完论文不许只写"读后感"——写一页**卡片**，格式固定（照 S11 §0 的四件套训练）：

```
┌─ Claim 卡片 ──────────────────────────────────────────────┐
│ 关系 R：{ (公开输入, witness) } 满足什么谓词              │ ← 用 S03/S06 的语言写
│ 协议骨架：P 发什么 / V 查什么 / 挑战在哪派生              │ ← 逐轮，不省略
│ 健全性主张：逃逸概率 ε 与它依赖的每个假设                 │ ← M06 预算表格式
│ 工程假设：域大小/承诺对象/验证方能本地算什么              │ ← S10 虚拟主张原则
│ 我方疑问：论文没说清的三处                                │ ← 给作者的提问
└──────────────────────────────────────────────────────────┘
```

验收：卡片能让**没读过论文的人**实现协议——做不到就是没读懂。以 LogUp 为例，卡片的关系一栏应能写出 S09 §3 的恒等式与多重度公式（对照检查你的卡片）。

2. Step 2 · 最小实现（范围裁剪的艺术）

死法排行：一上来就完整实现（死在第 1 周）、直接在真实域上调试（死于无法定位）、过早优化（死于偏离正确性）。裁剪三原则：

1. **钟表先行**：先在 mod-p 小域（甚至 mod 13）实现协议核心，全部手算对拍（S02–S09 的做法）；
2. **引擎复用**：sumcheck/承诺/eq 直接用已有轮子（P2/P5/P7 的 toy，或仓库引擎）——你实现的是**新协议层**，不是发动机；

3. **接口对齐目标仓库**：若目标是给本仓库提 PR，先读 L07 的精读路线 + FPGA 指导的对应算子规格——上游的接口约定（如 `ProvableOp` trait）是你的交付形状。

时间预算：卡片 1 天；toy 3–5 天；真实域移植 2–3 天；对拍与 fuzz 2 天；PR 1 天——**两周一个协议**是这个手艺的正常速度。

3. Step 3 · 三板斧验证

1. **Golden 对拍**：中间值逐点 dump 比对（L07 §2 的工具箱）——对拍不过 = 实现 bug 或论文理解错，二分定位；
2. **Fuzz**：随机表/随机挑战/随机参数 + **负例族**（S06 P2 纪律）：每个"应拒绝"的作弊构造写成回归测试（篡改表、篡改轮消息、错位挑战、跨陈述重放——M06 §2 的攻击全做成负例）；
3. **性能 sanity**：复杂度账单对照（S06 §3 的方法）——比账单差 10× 以上说明有结构性问题（多余拷贝/错误数据布局）。

4. Step 4 · 上游 PR

1. fork → 分支（`feat/xxx-toy` 或 `docs/xxx-claim-card`）→ **小步提交**（每步可独立 review）；
2. PR 描述 = claim 卡片 + toy 验证结果 + 对拍命令——让 reviewer 三分钟内能复现你的正确性；
3. 面对review：把每条意见转成卡片更新或测试用例；**被指出理解错误是这门手艺的正常反馈回路**（S 课 v1→v2 就是先例）。

5. 常见死法与急救

死法	症状	急救
论文有错/省略	按论文实现对不上	回到 claim 卡片标注疑问；用健全性直觉定位（哪个检查在保护哪个性质）；邮件/issue 问作者
性能差 100×	toy 快、真实域慢	域化/拷贝/内存布局逐项剖析（L07 §0 的 measure）
负例意外通过	应拒绝的构造被接受	先查 transcript 绑定（M06 §2）——十有八九是它

死法	症状	急救
上游风格不合	PR 被要求重写	接口对齐先行（Step 2 第 3 条）

6. 本讲任务

1. 从 S11 精读的协议中选一个（建议：LogUp 或 batched sumcheck 的论文版 Protocol 2），写 claim 卡片；
2. 完成最小实现（mod-p toy + 真实域移植）+ 三板斧；
3. 提 PR（本仓库、dp-crypto，或任何公开 ZK 仓库）——**PR 链接即本讲交付物**。

下一讲：`M08 顶石设计`——把全部能力投进一个陌生负载的完整系统设计，三位模拟评审答辩，出师。

M08 · 顶石设计：给陌生负载造一套完整证明系统（出师关）

> M-track 的收官：给你一个**没见过的负载**，两周内产出 15 页设计文档 + 30 分钟答辩，接受三位"模拟评审"（导师扮演三种恶评人格）。这是 L4 的毕业考，也是 L5 的入场券——评审意见的质量会告诉你离 L5 还有多远。 前置：M01（地图）、M06（预算表/审计清单）、S10（系统全景）、论文 §4（定制协议的范例）。

---

1. 设计文档骨架（15 页的固定结构）

页	章节	必须回答的问题	对应课程
1	负载与威胁模型	证什么？谁在作弊？P/V 的算力与信任假设？	S01
2	计算剖析	算子分布（线性/非线性/位运算占比）、数据量公式、值域上界	M03 三问
3–4	算术化选型	四方言对比 + **定量**理由（门数/约束数估算）	M01 轴一

页	章节	必须回答的问题	对应课程
5–6	PCS 与查表选型	决策树走一遍；"为什么不选 X"逐条	M01 轴二/三
7–9	协议细节	逐算子 PIOP（复用已学协议 / 新设计）、claim 链图、transcript 吸收序列	S06–S09
10	健全性预算表	M06 §1 格式，每个数字有出处	M06
11–12	工程计划	数据流图、内存/带宽预算、瓶颈预判、硬件路线	FPGA 指导
13	语义边界	精度/溢出/范围外的假设与兜底	M06 §4
14–15	实验计划	要测什么、预期数字、失败判据	论文 §5 的读法反向使用

2. 三个候选题目（也可自提，需导师认可负载的代表性）

T-A · 70B 模型推理的手机验证：模型在服务器、验证方是手机。关键张力：内存（M03 的域选择 × 内存预算表）、证明大小（手机带宽）、验证时间（<1 s？）——HyperKZG vs WHIR 的选型是本题的核心考点；70B 的 witness 量级要用 M00 的公式外推并给出内存方案（空间高效 PCS？折叠流式？）。

T-B · EVM 区块执行证明：负载 = 交易签名验证 + 状态读写 + EVM 字节码。关键张力：VM 负载指向 AIR + Lasso 系查表（M01/M04），与 LLM 路线完全不同；签名验证（ECDSA/Ed25519）是查表/MSM 的复合考点；聚合多区块时的折叠（M05）。

T-C · 隐私交易（隐藏金额的转账正确性）：与 DeepProve 相反的威胁模型——**要零知识**（隐藏输入）+ 范围证明（金额非负 → 查表/位分解）。考点：ZK 变体的代价、Pedersen 承诺与 MLE 承诺的混合、以及"证明范围之外"的语义（M06 §4）。

3. 三位模拟评审（导师扮演，各具人格）

人格	攻击风格	典型问题
理论洁癖	只攻健全性与假设	"这个恒等式在扩展域上还成立吗？""你的 RBR 预算把 grinding 算进去了吗？""第 7 页的 eq 表维度和第 9 页对不上。"
工程现实	只攻成本与落地	"这个数字的硬件依据？""瓶颈预判的依据是测量还是想象？""这套协议需要新算子——团队三个月能写完吗？"
安全偏执	只攻边界与信任	"statement binding 在哪？""随机种子谁提供？""这个'证明范围之外'的假设被谁兜底？"

应对纪律：每条质疑要么当场推导回答，要么登记为文档修订——**含糊其辞 = 该关不通过**（M00 出师标准的评审关）。

4. 答辩流程

1. 提前 48 小时交文档（评审预读）；
2. 30 分钟：20 分钟主讲（负载 → 选型 → 协议 → 预算 → 实验计划）+ 10 分钟三评审轮番质询；
3. 评分：选型定量性 / 协议完整性 / 预算表可信度 / 质询应对 四项各 25%；
4. 未过：按评审意见修订，两周后重答辩——**这是 M00 承诺的"真实战场"形态**。

5. 出师之后

- 与导师转为同行关系：互评设计、共读 ePrint、合写短文；
- 持续循环：每周论文筛读 → 每季度开源贡献 → 每年一次"自己一年前作品"的批判性重读；
- 行业顶尖不是证书，是**被同行反复挑战后仍然站立的记录**——本课程给你的是进入那个循环的资格。

M-track 全部完成。回到 L09 §5 的路标，上路。